

Diagrams capitolul 5

Acest fisier contine cod Mermaid pentru diagramele recomandate in sectiunea 5.2. Diagramele pot fi randate si exportate ca imagini, apoi introduse in LaTeX in locul placeholderelor.

Figura 5.6 - Fluxul de evenimente in canvas

```
flowchart LR
    User[Utilizator] --> Input[Eveniment mouse sau tastatura]
    Input --> Events[utils/canvas_helpers/canvas_events.py]
    Events --> Canvas[AnnotationCanvas]
    Canvas --> Manager[AnnotationManager]
    Manager --> Item[Item grafic]
    Item --> Scene[QGraphicsScene]

    Events -->|zoom/pan| Canvas
    Events -->|rect/poly/brush/eraser| Manager
    Canvas -->|addItem/removeItem| Scene
```

Figura 5.7 - Clasele principale pentru adnotari

```
classDiagram
    class AnnotationManager {
        +add_rect()
        +add_poly()
        +add_mask()
        +remove()
        +clear()
        +load_annotations()
        +get_all_dicts()
    }

    class BoundingBoxItem {
        +annotation_id
        +label
        +color
        +to_dict()
        +paint()
    }

    class PolygonItem {
        +annotation_id
        +label
        +color
        +to_dict()
        +paint()
    }
```

```

}

class MaskItem {
    +annotation_id
    +label
    +color
    +paint_brush()
    +erase_brush()
    +to_dict()
}

class QGraphicsRectItem
class QGraphicsPolygonItem
class QGraphicsPixmapItem

AnnotationManager "1" o-- "*" BoundingBoxItem
AnnotationManager "1" o-- "*" PolygonItem
AnnotationManager "1" o-- "*" MaskItem

QGraphicsRectItem <|-- BoundingBoxItem
QGraphicsPolygonItem <|-- PolygonItem
QGraphicsPixmapItem <|-- MaskItem

```

Figura 5.8 - Fluxul AutoSeg YOLO in client

```

flowchart LR
    User[Utilizator] --> Button[Buton AutoSeg]
    Button --> Dialog[AutoSegRunDialog]
    Dialog --> Settings[Citire QSettings]
    Settings --> Worker[AutoSegWorker]
    Worker --> Process[Proces YOLO extern]
    Process --> Json[Listă JSON de detectii]
    Json --> Handler[on_autoseg_yolo_result]
    Handler --> Manager[AnnotationManager]
    Manager --> Canvas[AnnotationCanvas]

    Handler -->|box| Rect[BoundingBoxItem]
    Handler -->|coordinates| Poly[PolygonItem]
    Handler -->|mask_coords| Mask[MaskItem]
    Rect --> Canvas
    Poly --> Canvas
    Mask --> Canvas

```

Figura 5.9 - Fluxul AutoMask pe baza unui punct selectat

```

flowchart LR
    User[Utilizator] --> Tool[Selectare AutoMask]
    Tool --> Click[Click in canvas]

```

```

Click --> Signal[autoseg_requested]
Signal --> Worker[AutoMaskWorker]
Worker --> Process[Proces extern Mask2Former sau segmentare]
Process --> Json[Obiect JSON cu coordinates]
Json --> Handler[on_autoseg_result]
Handler --> Canvas[AnnotationCanvas]
Canvas --> Manager[AnnotationManager]
Manager --> Mask[MaskItem]
Mask --> Scene[QGraphicsScene]

```

Figura 5.12 - Fluxul de incarcare imagine si protectia modificarilor nesalvate

```

flowchart TD
    Select[Selectare imagine noua] --> Check{ _unsaved_changes? }
    Check -->|Nu| Load[Incarcare imagine in canvas]
    Check -->|Da| Prompt[Dialog Save / Discard / Cancel]

    Prompt -->|Save| SaveJson[ToolBarPanel.save_json()]
    SaveJson -->|Succes| Confirm[LeftPanel.confirm_switch()]
    SaveJson -->|Esec sau anulare| Cancel[LeftPanel.cancel_switch()]

    Prompt -->|Discard| Load
    Prompt -->|Cancel| Cancel
    Confirm --> Load
    Load --> Canvas[AnnotationCanvas.load_image()]
    Canvas --> Manager[AnnotationManager.clear()]
    Canvas --> Status[Status bar actualizat]

```